PLATFORM-BASED SOFTWARE ENGINEERING · MEETING 2

Week 2 Worksheets

Contract-First Specification

This is a Word file, and you are meant to type in it. **Every shaded box and every empty table cell is somewhere you write.**

Fill it in as you work through the Reading Guide and the Lab. Each worksheet says at the top where it belongs.

When you finish, save it as a PDF (**File → Save As → PDF**) and commit both the Word file and the PDF to your branch.

> **The five worksheets**
>
> **W0 — Meeting 1 catch-up.** Do this first if your team has no domain, no client table or no repository.
>
> **W1 — Resource model.** After Section 2.2. Worth 30% of A1.
>
> **W2 — Method, safety and idempotency.** After Section 2.5.
>
> **W3 — Error catalogue.** After Section 2.6. Worth 25% of A1.
>
> **W4 — State machine.** After Section 2.9.

Fill this in before you start.

Name	Gibran
Student number	536807
Team	
Domain	
Branch (week2/…)	
Date	Aug 28, 2026

W0 · Choose a system to practise on

About 20 minutes. Do this before the Reading Guide. Everyone does it.

You work **on your own** this week. Groups are formed next meeting, and your group will choose its real domain then.

So you need something of your own to practise on. It does not have to be the system you build all semester. It has to be **something you can describe in a paragraph and reason about**.

Pick anything small and real:

- Renting lockers in a campus building
- Booking a badminton court
- Borrowing equipment from a lab
- Ordering a print job at a campus print shop
- Reserving a study room in the library

Or use **KANTIN** from the handbook, if nothing comes to mind. That is allowed this week. You will learn more from a system you have actually stood inside, but a working file about KANTIN beats an empty file about something original.

Step 1 — Write it in one paragraph

Not a feature list. One paragraph of ordinary prose: **who does what, in what order, and what can go wrong.** Something you could read aloud in thirty seconds.

> A customer views available badminton courts, selects a date and time, and makes a booking through the system. Court staff manages court availability and handles bookings at the facility, while an administrator manages the courts, staff, and system settings. The system must prevent two customers from booking the same court simultaneously, and it must handle situations where court staff temporarily loses network connectivity while updating information.

Step 2 — Check it against four conditions

These are the same four conditions your **group** must satisfy next meeting. Practising them now means you can argue for a good domain then, instead of accepting whatever gets suggested first.

Your practice system does not have to pass all four this week. Notice which ones it fails, and why.

Condition	Yes?	Why it matters later
Three or more kinds of user, allowed to do different things	Yes	One kind of user leaves Chapter 4 with no permissions to build
At least one action that must not happen twice — money, a booking, anything you cannot undo	Yes	This is the operation you protect in this week's lab
At least one user who works without a reliable network	Yes	Otherwise Chapter 6 becomes a pretend exercise
Small enough to finish — one workflow, start to end	Yes	Groups that pick "a social network" build fourteen half-features and fail Chapter 14

> **The second condition is the one you cannot skip this week.**

Condition	Yes?	Why it matters later

> The whole lab is built around the action that must not happen twice. If your practice system has none, pick a different system now. It takes two minutes, and without it half of Chapter 2 has nothing to bite on.

Step 3 — Name your actors

Who or what talks to this system? Three or four is enough.

Try to include one that is **not a person** — a display screen, a sensor, a scheduled job, an AI assistant. Chapter 1 explains why that matters.

Customer – views available courts and books/cancels their bookings

Court staff – manages court availability and handles bookings at the facility

Administrator – manages courts, staff, and system-level settings

Step 4 — Make your own GitHub repository

This is how you hand in. You submit a link, not files. Everything you do this week lives in a repository of your own.

You will use Git and GitHub every week for the rest of this course, so it is worth the fifteen minutes now rather than in a hurry later.

4a. Get an account

If you do not already have one, sign up at **github.com**. Free is fine.

Use your **university email** if you can, and then apply for the free GitHub Student Developer Pack at `education.github.com` — it costs nothing and gives you private repositories and other tools all semester.

4b. Create the repository

On github.com, click **New repository** (the green button, or the + at the top right). Fill in:

Field	What to put
Repository name	`pbse-week2-<your-name>` — for example `pbse-week2-rizky`
Description	One line: the practice system you chose
Public or Private	**Public** is simplest. Choose Private only if you add me as a collaborator, or I cannot mark it
Add a README	**Yes**, tick it. It gives you something to clone
.gitignore	Choose **Node** from the dropdown

Then click **Create repository**.

> **Never put a password, an API key, or a `.env` file in a repository.** Once it is in the history, it is public forever, even if you delete the file afterwards. This week you have no secrets to leak — good habits start before you need them.

4c. Get it onto your computer

Click the green **Code** button, copy the HTTPS address, then in a terminal:

```
git clone https://github.com/<your-username>/pbse-week2-<your-name>.git
cd pbse-week2-<your-name>
mkdir spec
```

The first time you push, Git asks you to sign in. A browser window opens — sign in there and it remembers you afterwards.

Expected result. A folder on your computer with a `README.md` in it, and

`git status` says `On branch main` with nothing to commit.

4d. Write down what it is

Open `README.md` and replace it with four lines:

```
# PBSE Week 2 - <your name>

Practice system: <one sentence>
Interface: spec/openapi.yaml
Run the mock: cd spec && npm install && npm run mock
```

Then:

```
git add -A
git commit -m "week 2: my practice system"
git push
```

Refresh the page on github.com. Your README should be there.

If `git push` is rejected, you are probably not signed in yet. Run

`git push` again and complete the browser sign-in when it appears.

This repository is **yours alone**. Next meeting your group creates one repository for the group, and you bring across whatever is worth keeping.

What happens next meeting

All of this happens **in class**, with your group. Do none of it now. It is here so that none of it is a surprise.

- **Groups of four are formed.** The group is fixed for the semester.
- **The group chooses one domain**, checked against those same four conditions.
- **Four jobs are assigned** — interface owner, service owner, client owner, integration owner — rotating every three meetings, and everybody holds the interface job once.
- **One repository is created** for the group, with the folder structure in section 6 of the Week 1 Recap, and everyone commits to it under their own name.

Read section 6 of the recap so you know what is coming. Do not act on it yet.

Done?

- ☐ One paragraph describing a system
- ☐ Checked against the four conditions — and it definitely has an action that

must not happen twice

- ☐ Actors listed, including one that is not a person if you managed it
- ☐ A GitHub repository called `pbse-week2-<your-name>`, cloned to your

computer, with your README pushed to it

W1 · What things exist in your system

Do this after Section 2.2, before the lab. About 15 minutes.

You need this one to do the lab: it decides what goes in your file.

(The handbook calls these things resources, and calls this worksheet's job resource modelling.)

Step 1 — List every noun

Take your paragraph from Worksheet W0 and go through it marking **every noun**.

Do it on the actual text, not from memory. Working from memory skips the interesting cases, and the interesting cases are the ones worth marks.

List them all before you test any of them. Ten or so is fine.

1. Customer
2. Badminton Court
3. Date
4. Time
5. Booking
6. System
7. Court staff
8. Court availability
9. Facility
10. Administrator
11. Staff
12. System settings
13. Situation
14. Network connectivity
15. Information

Step 2 — Test each noun against three questions

A noun becomes a **resource** — a thing with its own address — only if the answer is yes to all three.

- **Can you point at one and name it?** An order has an ID you can put in an address. "The checkout process" does not.
- **Does it still exist after the request that made it?** An order does. An error message does not.
- **Can it change on its own?** A shop's opening hours change while the shop stays the same shop.

> **The reasoning column is what gets marked. The verdict on its own is worth nothing.** "Order = resource" earns zero. "Order is a resource because it has an identifier, it survives the request that created it, and its status changes without the outlet changing" earns full marks.

Noun	Can you name one?	Does it last?	Changes on its own?	Verdict	Why — and its address
Badminton court	Yes: Badminton court #5	Yes: the court continues to exist after created	Yes: its availability can change	Resource	Badminton court #5 is a specific court, status can change from available to booked. address: /courts/{id}
Booking	Yes: Booking #123	Yes: the booking remains after the request creates it	Yes: it can go from pending to confirmed to cancelled/completed	Resource	Has its own identify and lifecycle Address: /bookings/{id}
Booking Process	No: there is no specific “Booking process #123”	No: it’s a sequence of actions rather than a persistent thing	No: It describes what happens, rather than an object with its own state	Not a Resource	It is a process. No address

For this week you need two things that pass, and one that you reject. The third thing and the rest of the rejections come with A1.

If you rejected nothing at all, you did not list the nouns honestly. Every system has at least one process that sounds like a thing.

Two verdicts worth watching for:

- **Something that belongs inside something else** *(the handbook says sub-resource)*. It means nothing on its own and nobody ever asks for one by itself. In KANTIN, a single line of an order. Put it in a list inside the order, not at `/order-lines/{id}`.

- **A process, not a thing.** In KANTIN, "checkout" — a sequence of operations on orders and payments. There is no `/checkouts/{id}` anybody would ever fetch.

Step 3 — The screen-shaped endpoint you gave up

From Checkpoint 2 of the Reading Guide. Almost everybody has one.

The endpoint I would have designed: GET/dashboard *for example, GET /home*

What it returns: A summary of available courts and the customer bookings

The real resources it is built from: Courts and bookings

Web client - Dashboard

Mobile client – Home screen

Admin client – Management screen

AI client – “Whats available today?”

> **Why the screen-shaped version fails.**
>
> It works — for one client. Then a second client needs two of those three things but not the third. Then a third client needs one of them in a shape you never imagined.
>
> Each new client adds a field, or gets its own screen-shaped endpoint. Within a month your service has fourteen endpoints returning overlapping pieces of the same four resources. None can be cached. None can be reasoned about. And all of them have to change together.
>
> **The rule: resources come from the domain. Screens are built by the client, out of resources.**

Step 4 — Addresses and IDs

Write the address of your two things, and pick an ID prefix for each. Check each address against the five rules as you write it.

`ord_51Fa93cD`, never `1`. A plain counting number tells a competitor how many orders you get per day, invites somebody to try `/orders/1` through

`/orders/5000`, and ties your public addresses to one database forever.

Thing	Address	ID prefix	Example ID
Court	/v1/courts/{id}	court_	court_3Kq92s
Booking	/v1/bookings/{id}	book_	book_A8wdj2

You now have everything you need to start the lab. Go to **Document 3**.

W2 · Which operations are safe to repeat

Do this during Step 7 of the lab. About 10 minutes.

(The handbook says idempotent for "safe to repeat", safe for "read-only", and idempotency key for the one-off ticket number that makes a repeat harmless.)

Step 1 — Find the operation that must not happen twice

Go through the operations you have in mind and ask this about each one:

> **If the network drops and the client sends this again, what does the user lose?**
>
> "Nothing" → it is already safe to repeat. It needs no protection.
>
> "Money, a duplicate record, or a second notification" → it needs a ticket number.
>
> There is no third answer.

Operation	Method	Safe to repeat?	What sending it twice costs the user
List badminton courts	GET	Yes	Noting, it only reads count data
Create badminton court booking	POST	No – needs idempotency key	Could create a duplicate booking
Get a badminton court	GET	Yes	Nothing, it only reads count data
Retire badminton court	POST	Yes	Nothing serious, your contract says repeating it returns the exiting retirement record

Reference — the five methods

Method	Read-only	Safe to repeat	Can a client that lost the network just send it again?
GET	yes	yes	Yes. Nothing changes
PUT	no	yes	Yes. PUT means "make this thing look exactly like this". Doing that twice is the same as once
DELETE	no	yes	Yes. The second call returns 404 or 204. Either is fine, and the state is the same
PATCH	no	**not always**	Only if the patch is absolute. `{"quantity": 3}` is safe to repeat. `{"quantity": "+1"}` is not, and turns two portions into four
POST	no	no	**No.** Each call creates a new thing — unless you add a ticket number

Check your PATCH rows carefully. A patch that says "add one more" rather than "make it exactly 3" is the trap here.

Step 2 — Your version of the lift story

The handbook's story is Rizky in a lift with one bar of signal, 23,000 rupiah of nasi goreng, and a `201` that never arrives.

Write yours in **two or three sentences**: who is doing what, where the network dies, and what it costs them when the app tries again.

This is not creative writing. It is how you check that your system really has a dangerous operation at all.

> A student uses the badminton court booking app to reserve a court for a specific time slot. While confirming the booking, the student's network connection drops before the booking confirmation reaches the app. The student retires the booking, and without protection the same court could be booked twice, creating a duplicate booking and potentially causing a conflict for the student.

Remember why this cannot be fixed with better error handling: a client that loses its connection **cannot know** whether its request was processed. That is how networks work. You cannot remove the uncertainty — you can only make trying again harmless.

Step 3 — The four sentences

Writing "we use idempotency keys" earns zero. You need all four sentences, in their final wording. Write them here, then paste them into the header's

`description` in your `openapi.yaml`.

1. The header name, and what the value looks like

The Idempotency-Key header is required for booking requests, and its value must be a version-4 UUID with hyphens

2. Which operations require it

It's required on POST /v1/bookings, because creating a badminton court booking is a consequential operation that must not happen twice.

3. How long the server remembers a ticket number — an actual number

Keys are kept for 24 hours. A key reused after that period is treated as new

4. What happens if the same ticket number arrives with different contents

The server returns 409 Conflict with the problem type https://api.example.com/problems/idempotency-key-reuse

> **Later, not this week.** Your file also has to say what the server does in each of the four cases in Table 2.5 — especially the third one, *same number, different contents*, which is the row teams forget and the row a marker tests. That goes in with A1.

Step 4 — Where the ticket number is created

Fill in the blank, and mean it:

> The client creates a fresh random ID at the moment when the user confirms the booking then stores it with the pending request, and reuses it unchanged for every retry of that same intention.

Two ways to get this wrong:

- **Making it inside the function that sends the request.** Every retry gets a new number, so the server sees three unrelated orders. The number does nothing at all.
- **Making one and keeping it for the whole session.** Now it is reused across genuinely different actions, and everything after the first one is refused with `409 idempotency-key-reuse`.

In Chapter 6, this waiting request becomes a real row in a small database on the phone, and the ticket number becomes a column of that row. **Design it now, so that Chapter 6 adds to your work instead of replacing it.**

W3 · Your list of errors

> **Not due this week.**
>
> This worksheet is for assignment **A1**, due at Meeting 4. Do not do it now.
>
> This week the lab gives you one shared `Problem` shape, and that is enough. Come back here when you start A1.

About 20 minutes, when you get to it. 25% of A1 is for writing down the ways things fail as carefully as the ways they work.

(The handbook calls this list your error catalogue, and calls your own extra fields extension members.)

> **What is assessed**
>
> Every failure reply your service can send must appear in `openapi.yaml`, with its `type` address and a worked example.
>
> "An error we were never told about" is one of the four kinds of fault the other team can report against you in Meeting 7, and it is the most common one every year. The teams that lose the fewest marks there are the ones who took this seriously in Chapter 2.

Step 1 — Sort failures into three kinds

Ask the three questions in order. The first "yes" gives you the kind.

Kind	Question	Codes	What the client should do
The client got it wrong	Did the client send something wrong?	400, 401, 403, 404, 422	Do not send it again. Fix the request, or ask the user to do something
A business rule said no *(handbook: domain rejection)*	Was the request fine, but not allowed right now?	409, 422	Do not send it again. Show the reason to the user. This is a normal answer, not a bug
Our service broke	Did our service fail?	500, 502, 503, 504	Try again, waiting longer each time. Never show the user the internal details

The middle row is where student services go wrong, in two opposite directions.

Using `500` when a business rule said no tells every client to keep retrying something that can never work. It also buries normal business events inside your error alerts.

Using `200` with `{"success": false}` is worse. From the outside, everything looks fine. Retries, monitoring, caching and the other team's error handler all treat it as a success. And every client now has to check a field in the body on every single reply, forever. Forget once, and a failure is silently treated as a success.

Step 2 — Your catalogue

One row per failure your service can send. This table goes into A1 as it is.

`type` address (never changes once published)	Status	Kind	When it happens	Your extra fields	What the client should do
`…/problems/illegal-transition`	409	rule said no	Any move not in your W4 table	`from`, `to`, `allowedFrom`	Do not send it again. Update the buttons from `allowedFrom`
`…/problems/idempotency-key-reuse`	409	client	Same ticket number, different contents	`key`	Fix the client. Do not send it again with that number
`…/problems/validation-failed`	422	client	Correctly formed, but the values make no sense	`errors[]` with `field` and `reason`	Correct the form and send again

Six rows is a reasonable minimum. If you have fewer than five, you have only written down the failures you happened to think of — which is exactly how this part of the marking goes wrong.

`type` is the one that must never change once you publish it. Client code looks at that exact text to decide what to do.

`title` is always the same for a given `type`, and it is written for a developer reading a log file, not for the end user.

`detail` describes this one particular failure. It must never contain a stack trace, a piece of SQL, or the name of an internal server. It goes to strangers.

Step 3 — Design one extra field properly

The standard (RFC 9457) gives you five fields. You are allowed to add your own on top of them. Those extra fields turn an error that is merely correct into one a client can actually act on, and they are marked in A1.

Study the handbook's example before you write yours:

```
{
  "type": "https://api.kantin.example/problems/item-unavailable",
  "title": "A requested menu item is unavailable",
  "status": 409,
  "detail": "Ayam geprek (itm_3Bn) sold out at 12:14 local time.",
  "instance": "/v1/orders",
  "unavailableItems": ["itm_3Bn"],
  "outletId": "out_7Kq",
  "suggestedAlternatives": ["itm_9Lp", "itm_2Rt"]
}
```

The last three fields are not in the standard. They were invented for this system. `unavailableItems` lets the app grey out exactly the right line instead of rejecting the whole basket.

`suggestedAlternatives` lets it offer something else instead of an apology.

Now write yours — the whole document

What can the client do with your extra fields that it could not do without them? Answer in one sentence. If the answer is "nothing", they are decoration and you have not done this step.

Step 4 — The two wrong versions

Write the wrong version of one of your own failures, both ways. Then you will recognise them when somebody in your group proposes one.

Wrong — a failure dressed up as a success

```
HTTP/1.1 200 OK    { "success": false, "message": … }
```

Wrong — a business event dressed up as a server crash

```
HTTP/1.1 500 Internal Server Error    { "error": … }
```

Correct — the status code carries the meaning

```
HTTP/1.1 409 Conflict    Content-Type: application/problem+json
```

With the correct version, everything sitting between you and the client behaves properly without knowing anything about your system. And the client can show the user any wording it likes, in any language — which matters, because your `title` is in English and your users may not be.

Step 5 — Into the file

- ☐ The `Problem` shape written **once**, under `components/schemas`
- ☐ At least one reusable error reply under `components/responses`
- ☐ Every operation lists **every** code it can send, not just 200 or 201

- ☐ Every non-2xx has a worked `example`
- ☐ The content type is `application/problem+json`, not `application/json`

W4 · Which moves are allowed

> **Not due this week.**
>
> This worksheet is for assignment **A1**, due at Meeting 4. Do not do it now.
>
> This week, the fixed list of statuses you put in your file during lab Step 6 is enough. Come back here when you start A1.

About 20 minutes, when you get to it.

(The handbook calls this map of statuses and moves a state machine.)

Everything in Chapter 2 so far describes your system **one request at a time**. Nothing yet says which **order of requests** is allowed.

Can an order go from `ready` back to `placed`? Can somebody cancel food that is already cooking? Right now, the answers live only in your team's heads — and Meeting 7 exists to prove that a stranger cannot read your team's heads.

Step 1 — List your statuses

Split them into two groups. A **final** status never changes again. *(The handbook calls these terminal.)*

Statuses something can still move out of

Final statuses

KANTIN has eight statuses. Four are active — `placed`, `accepted`, `preparing`,

`ready` — and four are final: `collected`, `delivered`, `rejected`, `cancelled`.

Between the four active ones there are twelve possible moves. **Nine of them are wrong.** Your domain will have a similar ratio, and the wrong ones are the interesting part.

Step 2 — The transition table

Every move gets its own address, following Rule 5. **Any pair of statuses not in this table is not allowed, and returns `409`.**

This table is the whole of your domain's logic, and it fits on one page.

From	To	Caused by	Who may do it	Why it belongs here
—		POST /v1/…		The one that creates it. The only move that needs a ticket number

Two things to get right while you fill this in:

Name each move's address with a noun. …/acceptance, …/rejection,

…/preparation, …/readiness, …/collection. Not …/accept. Not

…/markReady.

Keeping the address a noun gives the move a body of its own (a reason), a reply of its own (a timestamp, a consequence), and a record you can fetch later with

GET. A verb in the address gives you none of that.

Mark the move you cannot undo. In KANTIN it is accepted → preparing: after that, the food exists. Chapter 13 will ask you to list these, and to refuse to let an AI agent do one unsupervised. You will be glad the list already exists.

Moves I cannot undo:

The last point at which cancelling still works:

Step 3 — Three rules that follow

All three are assessed.

3.1 One error type for every move that is not allowed

Every one of them is a business rule saying no, so they all get 409 with **one**

type that never changes — not one type per case, and not a different status code per case.

Put the specifics in extra fields, so the client checks one thing and then reads the detail:

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json
```

```
{ "type": "https://api.example.com/problems/illegal-transition",
  "title": "That status change is not permitted",
  "status": 409,
  "detail": "…", "from": "…", "to": "…",
  "allowedFrom": [ … ] }
```

`allowedFrom` is what lets a client grey out the button, instead of offering it and then apologising.

3.2 The client must be able to see the refusal coming

Which field, in which reply, lets a client that already has a copy of the thing — plus this table — work out the answer **without asking the server**?

Field:

In which reply:

> If you also publish a convenience flag such as `cancellable` or `editable`, that flag is now a **second statement of the same truth**. It must never disagree with the table.
>
> **Calculate it from the rules. Never store it.** A stored flag and a state machine will drift apart, and it is the flag that clients believe.

Convenience flags I publish, and how each one is calculated:

3.3 Repeating a final move is not an error

A courier's phone will send the same "delivered" twice. Chapter 6 guarantees it.

The second one must answer **`200`** with the record you already have, not `409`. What the caller wanted is already true, so there is nothing to refuse.

Keep `409` for a move that would take the thing somewhere it is not allowed to go.

This is the same idea as "safe to repeat" in Section 2.4, applied to statuses instead of methods.

The move most likely to arrive twice:

It returns 200 with:

Step 4 — Put the table in the interface, not in a chat

This table is **part of your interface**. It goes in three places inside

`openapi.yaml`:

- ☐ The statuses as a fixed list (`enum`) on the `status` field, with the rule

for values a client does not recognise

- ☐ The table itself in `info.description`, next to your money and timestamp

conventions

- ☐ The `illegal-transition` type in your error catalogue from W3, with a worked example showing `from`, `to` and `allowedFrom`

A table that lives in a WhatsApp group, a slide, or somebody's memory is **not** part of your interface. The Meeting 7 team will never see it.

Why this pays off four times

- **Section 6.6** asks how to settle a disagreement about `status` between the phone and the server. You can read the answer off this table instead of inventing it under pressure.
- **Section 12.5** asks you to write errors an AI agent can act on. `allowedFrom` is exactly that.
- **Section 13.4** asks you to sort every operation by whether it can be undone. This table is already that list.
- **Meeting 14** markers will ask you to justify one refusal. You will have the answer in writing.